



Agent Self-Evolution

Phase 4 Validation Report

Tool implementation code via iterative test-feedback repair

Date: June 25, 2026

Repository: github.com/jramos/agent-self-evolution

Executive Summary

Agent Self-Evolution is a standalone, API-only optimization pipeline that improves the artifacts an agent runs on and ships only the changes that prove out behind a noise-aware deploy gate. Phases 1–3 validated the gate as a mechanism on the instructions layer (skill files, tool descriptions, and system-prompt sections). Phase 4 turns to the **tool implementation code** itself — the one surface with a real, deploy-grade gradient on a capable agent — and asks a sharper question: can the loop repair a genuinely broken tool when it is handed the failing test as feedback?

The answer is yes, with an honest boundary. On real historical bugs harvested from an active repository’s git stream — each carrying its upstream fix commit as a **ground-truth executable oracle** — the loop produced a gate-passing, oracle-matching fix for **12 of 20** bugs (deploy-reachable **0.60**, Wilson 95% CI [0.39, 0.78]), corroborated by a second campaign at **34 of 46** (0.74, [0.60, 0.84]). Both clear a pre-registered futility floor of 0.10 (verdict GREEN). A leakage check shows the test’s expected values are load-bearing, so this is **test-feedback repair, not autonomous re-derivation**. The result is the product thesis: spend where a concrete executable test and real headroom exist; the gate certifies the fix didn’t regress.

Key Result	Value
Deploy-reachable (primary, N=20)	12 / 20 = 0.60 [0.39, 0.78]
Deploy-reachable (corroborating, N=46)	34 / 46 = 0.74 [0.60, 0.84]
Clears pre-registered futility floor (0.10)?	yes — verdict GREEN
Decision basis	executable oracle (fix-commit tests + behavioral match)
Cost (N=20 / N=46)	\$3.10 / \$12.01

Background

An agent’s behavior is governed by three layers. Phases 1–3 evolved the middle one (instructions). Phase 4 reaches the bottom one — the Python that implements each tool — because that is where text edits stop being redundant on a capable model and a real gradient appears.

Layer	What It Is	How Phase 4 Improves It
Model Weights	The underlying LLM (Claude, GPT, etc.)	Out of scope (RL training)
Instructions	Skills, system prompts, tool descriptions	Phases 1–3 (gate validated)
Tool Code	Python implementations of each tool	Phase 4: repair from a failing test

The campaign resolves the broader question — *does evolving an artifact change a capable agent's behavior?* — into three regimes. **GREEN**: where the signal is non-inferable and there is an executable oracle (code repair), there is a real gradient. **NULL**: where a task is binary-correct and the agent is already competent, there is no headroom, and the pre-flight refuses the run. **FRONTIER**: open-ended generative quality, where the judge saturates and an execution oracle has nothing targeted to bind to. Phase 4 lives squarely in the GREEN regime.

Approach: Iterative Test-Feedback Repair

Real historical tool bugs are harvested from an active repository's git stream, each carrying its upstream fix commit as a **ground-truth oracle**. The loop checks out the buggy parent in a throwaway worktree with an isolated venv, feeds the proposer the failing test's output, and asks for a whole-file repair. A repair counts only if it passes the full fix-commit test set and behaviorally matches the upstream fix. The unit of analysis is the organism (one bug); a bug is **deploy-reachable** when a majority of three independent seeds produce an oracle-matching fix.

The headline finding about engines is deflationary, and it matters: **the value is the signal and the gate, not the search**. Plain best-of-N resampling matches GEPA on this corpus, and a real population-based evolutionary search (the imbue darwinian_evolver) was tested at equal budget and **strictly dominated by best-of-N** — 6 of 23 bugs recovered versus 8 of 23, recovering nothing best-of-N missed.

Engine	What it does	Verdict
DSPy + GEPA	Reflective prompt evolution from traces	Validated for Phases 1–3
Best-of-N resampling	Sample K repairs, keep any that pass the gate	Matches GEPA here
Darwinian evolver (imbue)	Population-based evolutionary search	Tested, rejected (6/23 vs 8/23)
Test-feedback repair	Whole-file repair from the failing test, gated	Phase 4 — the shipped engine

Experiment

Parameter	Value
Proposer LM	gpt-5.4-mini class
Unit of analysis	one real historical bug (organism)
Seeds per organism	3
Deploy-reachable rule	majority of 3 seeds produce an oracle-matching fix
Bug corpus	one active repository's git stream (hermes-agent)

Oracle	upstream fix commit's full test set + behavioral match
Gate	held-out split, surface freeze, file scope, baseline-diff floor
Campaign size / cost	N=20 (\$3.10) and N=46 (\$12.01)

The two campaigns are independent harvests under the same protocol; the second ran to N=46 on a \$12 cap. Binning the 46 by the upstream fix's size answers "does it only fix one-liners?" — it does not.

Results

Estimand (organism-level)	N=20	N=46
Deploy-reachable	12 / 20 = 0.60	34 / 46 = 0.74
Wilson 95% CI	[0.39, 0.78]	[0.60, 0.84]
ICC	0.326	0.630
Clears futility floor (0.10)?	yes (GREEN)	yes (GREEN)
Cost	\$3.10	\$12.01

It is not just trivial bugs

Upstream-fix size	deploy-reachable	Tool size	deploy-reachable
<=5 LOC	5/5 = 1.00	<8k chars	4/4 = 1.00
6–20 LOC	5/6 = 0.83	8–25k	13/16 = 0.81
>20 LOC	24/35 = 0.69	>25k	17/26 = 0.65

The median upstream fix is **45 LOC** and 35 of 46 organisms are large fixes (>20 LOC), repaired at **0.69** — the rate is carried by substantial bugs and degrades gracefully with complexity, not off a cliff at triviality. The easy bins are tiny (5/5, 4/4), so the trend is solid but the exact small-bin rates are noisy.

The expected values are load-bearing (leakage check)

Feedback regime	Deploy-reachable (of 12)
Full traceback (control)	11 / 12 — reproduces the headline
No-leak (expected values withheld)	3 / 12

Re-running the 12 deploy-reachable organisms with the failing test's expected values withheld collapses the rate from ~0.92 to **3/12 (~0.25)**. So **0.60 is test-feedback repair leaning on the test's assertions, not autonomous re-derivation**: in production the failing test is always present, so 0.60–0.74 is the deployable rate and ~0.25 is the autonomy floor. A fuzzed differential probed whether the answer-informed fixes generalize: of the control successes, the **4** meaningfully testable on fresh inputs (never in the tests) **matched the oracle on every input, 0 divergences** — evidence they generalize rather than hard-code, though a thin slice.

Read the verdict honestly

The 0.10 line is a futility floor, not a success bar — deploy-reachable would have to collapse to 4/20 before it failed. The honest headline is the point estimate **0.60 [0.39, 0.78] on one repository, one proposer model**: promising, not settled. The per-seed distribution is bimodal (10 unanimous successes, 8 clear failures), so 0.60 hinges on the 2 organisms sitting exactly at 2/3, each one seed-flip from moving the verdict. The corroborating N=46 run tightens the lower bound, but ICC swung (0.326 vs 0.630), so seed correlation — and the precision that rests on it — is itself variable.

Safety and Guardrails

A green test can lie. The deploy gate is built to resist the specific ways an LLM-authored patch games one, and every repair runs in a throwaway worktree that never touches the real repository.

Guardrail	What it prevents	Status
Held-out split	a fix that passes only the test it was shown	enforced
Surface freeze	new functions / signatures smuggled in	enforced
File scope	edits leaking outside the one tool file	enforced
Baseline-diff floor	new failures vs. the upstream fix	enforced
Worktree isolation	mutating the real repository	enforced
Human PR review	auto-deploy without a human	required

An adversarial gate-gaming audit fuzzed candidate fixes that passed the gate to see whether any merely hard-coded the test inputs. Across the adjudicated cases it found **15 real fixes and 0 confirmed slips** on the oracle-rich bugs: where the oracle carries diverse test inputs, the gate forces generality. The honest caveat is that thin-oracle bugs are gameable in principle — trust tracks oracle richness, which is exactly what the gate measures.

Roadmap

Phase	Target	Engine	Timeline	Status
-------	--------	--------	----------	--------

Phase 1	Skill files (SKILL.md)	DSPy + GEPA	3-4 weeks	Validated ✓
Phase 2	Tool descriptions	DSPy + GEPA	2-3 weeks	Validated ✓
Phase 3	System prompt sections	DSPy + GEPA	2-3 weeks	Validated ✓
Phase 4	Tool implementation code	Iterative test-feedback repair	3-4 weeks	Validated ✓
Phase 5	Continuous improvement	Propose-only triage sentinel	2 weeks	Shipped ✓

Phase 4 establishes the first surface where self-evolution earns deploy-grade traction on a capable agent — tool code, fed a failing test, behind a gate that resists a lying green. Phase 5 builds the propose-only front-end that finds such bugs in a repository’s git stream.

Immediate Next Steps

- **Independent held-out product.** Only 3 of 8 pilot bugs had tests input-diverse enough for a truly independent held-out; build the diverse-test product so the gate's anti-gaming claim rests on independence, not the oracle sidestep.
- **Second corpus and proposer.** The headline is one repository, one mini-class model; a second of each de-confounds oracle-presence from headroom and task type.
- **Broaden oracle-bearing supply.** The clean (bug, failing test, upstream fix) supply is the binding constraint — widen it (see Phase 5) so the loop has more real work to do.